09 — CAP + HANA Sample Project: Architecture & Data Model

What it is: how the Python templates (sheets 01-08) map onto a real SAP **full-stack** project — CAP (Node.js) as the backend, HANA as persistence, deployed as one MTA. Pattern based on a real CAP + AI Core project structure.

Project Layout

```
my-ai-app/
├── db/
│   └── schema.cds          <- data model (tables, as CDS entities)
├── srv/
│   ├── design-time.cds      <- service definitions (entities exposed + actions/functions)
│   └── runtime.js           <- service implementation (business logic, AI Core calls)
├── app/
│   └── chat-ui/webapp/      <- SAPUI5 freestyle app (see sheet 10)
├── package.json              <- cds.requires: destinations, AI Core config
├── xs-security.json          <- XSUAA roles/scopes
└── mta.yaml                  <- deployment descriptor (see sheet 10)
```

Why this shape: CAP separates WHAT the data looks like (`db/schema.cds`) from WHAT the service exposes (`srv/*.cds`) from HOW it behaves (`srv/*.js`) — mirrors the Python templates' own separation (`connect_env.py` = data access, `chatbot_core.py` = behaviour).

Data Model (`db/schema.cds`)

```
entity Conversation {
    key cID : UUID not null;                // primary key, server-generated
    userID: String;                         // owner — used for row-level security
    title: String;
    to_messages: Composition of many Message on to_messages.cID = $self;  // 1-to-many, owned
}

entity Message {
    key cID: Association to Conversation;    // FK back to the parent conversation
    key mID: UUID not null;
    role: String;                           // "user" | "assistant"
    content: LargeString;
    creation_time: Timestamp;
}

entity DocumentChunk: cuid, managed {        // cuid = auto key+createdAt/By; managed = audit fields
    text_chunk: LargeString;
    embedding: Vector(1536);                 // CDS-level equivalent of sheet 04's REAL_VECTOR
}
```

Service Definition (`srv/design-time.cds`)

```
service ChatService @(requires: 'authenticated-user') {     // XSUAA-protected
    entity Conversation @(restrict: [
        {grant: ['READ','WRITE','DELETE'], where: 'userID = $user'}   // row-level security
    ]) as projection on db.Conversation;

    action getChatRagResponse(user_query: String, conversationId: String) returns RagResponse;
    function deleteChatData() returns String;
}
```

`entity ... as projection on db.X` exposes a DB table as an OData endpoint automatically — CRUD (`GET`/`POST`/`PATCH`/`DELETE`) comes for free, no controller code needed. `action` / `function` are custom operations you implement yourself in `runtime.js` (sheet 10).

HANA Persistence — CQL (CDS Query Language)

```
// This is CAP's ORM-style query builder — compiles down to parameterized SQL, like sheet 01/03's raw SQL:
await INSERT.into(Conversation).entries({ cID, userID, title });
const messages = await SELECT.from(Message).where({ cID_cID: conversationId }).orderBy('creation_time');
await UPDATE(Conversation).set`title = ${newTitle}`.where`cID = ${conversationId}`;
await DELETE.from(Conversation).where({ cID: conversationId });
```

Key Points

- CAP's **CQL** (`SELECT.from`, `INSERT.into`) is the Node.js equivalent of sheet 01/03's raw `hdbcli` SQL — same relational concepts, ORM-style syntax, auto-parameterized (SQL-injection safe).
- `Composition of many` = owned/cascading relationship (deleting the Conversation deletes its Messages); `Association` = a reference/foreign key without ownership.
- `@restrict` with `where: 'userID = $user'` is **row-level security enforced by the framework** — every generated query is automatically scoped to the logged-in user.
- `Vector(1536)` in CDS is the modeling-layer equivalent of `REAL_VECTOR(1536)` from sheet 04 — same HANA Cloud vector engine underneath, exposed at a higher abstraction level.
- Actions/functions (`getChatRagResponse`) are how you expose **non-CRUD** operations (like "call the LLM") through an otherwise auto-generated OData service.

Interview Q&A

Q: What's the difference between `Association` and `Composition` in CDS? A: Composition implies ownership — the child's lifecycle is tied to the parent (cascade delete). Association is a plain reference with no ownership implication.

Q: How does `@restrict` differ from checking `userID` manually in a Python Flask/FastAPI route? A: It's declared once at the model level and enforced by the framework on every query automatically — you can't forget to add the check in a new endpoint, unlike manual `if` checks scattered across handler functions.

Q: Why put an `action` in the service definition instead of just calling the LLM from the UI5 frontend directly? A: Keeps API keys/deployment IDs server-side only, lets you apply masking/filtering/auth consistently, and keeps the frontend a thin client — same separation of concerns as the Python templates keeping AI Core calls out of any Streamlit/CLI presentation layer.